Causal Diagnosis and Minimal Repair of Agent Security Policies

From counterfactual policy failures to executable, regression-aware repairs

Agent Security | Access Control | Program Repair | Causal Diagnosis

Parsa Oryani | Sharif University of Technology | Research interests: secure and trustworthy agentic AI

Research thesis

Tool-using agents rely on policies that decide whether an action should be allowed, denied, or escalated for approval. A faulty rule can admit an unsafe action, but a broad correction can also block legitimate work. This proposal asks when counterfactual evidence about the rules that changed a decision can be converted into one executable policy repair that satisfies all known safety and utility requirements.

The core hypothesis is deliberately comparative: causal diagnosis may help a bounded repair engine search more effectively, but it is not assumed to outperform direct symbolic synthesis. The research contribution is to characterize when local intervention witnesses are jointly realizable as a valid repair and to measure when that guidance is worth its cost.

Problem and scope

The initial setting contains one tool-using agent, one mediated outbound-action boundary, and a specificity-ordered policy with allow, deny, and ask outcomes. A request context records the tool action, provenance, trusted attributes, user authority, policy version, and active approval exceptions. Raw evidence and approval records are immutable during repair; only a bounded set of ordinary policy rules may be edited.

A repair must satisfy independently reviewed must-deny and must-allow cases, preserve ask behavior where required, and minimize unnecessary changes to permissions and policy structure. Snapshot replay is used to evaluate policy semantics under fixed evidence; later full-agent runs test whether a selected repair remains useful after the agent replans.

Closest-work positioning

Quantitative policy repair and Restricter already formulate least-change access-control repair and tightening as constrained optimization.[1][2] DirectFix shows that localization and synthesis can be integrated through MaxSMT rather than separated into stages.[3] AgentSpec demonstrates practical runtime enforcement for LLM agents.[4] Therefore, the intended novelty is not another policy language or another repair solver. It is the diagnosis-to-edit relationship for ordered agent policies, including shadowing, interacting rules, missing-rule cases, provenance-aware predicates, and explicit regression constraints.

Research questions

RQ1. Under equal total budgets, when does intervention-guided search improve qualified repair rate, anytime quality, or time to a target repair compared with direct and structural baselines?

RQ2. When do individually successful rule interventions fail to admit a single jointly valid repair because of ordering, shadowing, omissions, grammar limits, or conflicting requirements?

RQ3. How often do snapshot-valid repairs remain effective when the agent replans, chooses an alternative tool sequence, or encounters a held-out benign or adversarial task?

Method

1. Executable policy model. Implement a deterministic interpreter for the declared policy subset. It evaluates every rule condition, applies explicit priority and first-match semantics, and returns allow, deny, or ask. Tiny instances are checked exhaustively.

2. Bounded repair grammar. Permit rule insertion, deletion, and condition edits over a typed Boolean grammar with fixed depth and patch-size limits. A MaxSMT objective first satisfies hard requirements, then minimizes legitimate-use loss, permission drift, and canonical syntax change.

3. Counterfactual diagnosis. Override one or two rule-eligibility decisions while holding raw evidence fixed. Record minimal intervention sets that change the target decision. These are model-relative influences, not claims about historical causality.

4. Realizability and fallback. Test whether one permitted patch confined to a diagnosed scope satisfies every hard requirement. Expand the scope when needed and retain a full-space direct solver as the correctness-preserving fallback. All diagnosis, feasibility, synthesis, and validation work is charged to the same budget.

5. Independent validation. Execute candidate patches in a second interpreter, evaluate hidden requirements, and rerun selected agent tasks. A timeout returns a validated incumbent with minimality unresolved; only complete bounded solving can certify optimality or infeasibility.

Expected technical artifact

The prototype will expose a small policy language, deterministic replay harness, intervention analyzer, direct and guided synthesis modes, and a machine-readable repair receipt containing the changed rules, satisfied requirements, permission impact, and unresolved assumptions.

Evaluation plan

The study begins with semantic fixtures for shadowing, precedence, interacting rules, missing rules, ask preservation, inconsistent requirements, and unexpressible repairs. It then uses independently authored faulty policies and held-out requirements grouped by scenario family. Mutation history may support diagnosis checks, but it is not treated as proof that one repair is uniquely correct.

Core comparisons share the interpreter, repair grammar, objective, evidence, solver, validator, and wall-clock budget: full-space symbolic synthesis; inexpensive ordered dependency analysis; spectrum or coverage localization; random scopes; and intervention guidance. The primary endpoint is hidden qualified repair rate: all hidden hard requirements pass and a predeclared legitimate-use floor is met. Secondary measures include unsafe allows, false denials, ask fidelity, permission drift, patch size, latency, solver effort, and the fraction of useful repairs outside preferred scopes.

A small full-agent study compares the original, direct-repair, and guided-repair policies on benign and adversarial tasks. Task completion, unsafe effects, approvals, latency, and alternative paths are reported separately from snapshot authorization accuracy.

Expected contributions and limits

Expected contributions are: (i) a precise model of diagnosis-to-edit realizability for a bounded ordered policy language; (ii) a reproducible comparison showing where causal guidance helps, is neutral, or wastes budget; and (iii) an auditable repair prototype with explicit regression constraints. A negative result - that direct synthesis dominates within credible scales - remains informative but is not presented as a guaranteed publication outcome.

The guarantees are conditional on complete requirements, faithful policy semantics, and trusted evidence. Fixed-context replay does not predict every replanning behavior. Incomplete predicates can make a needed repair inexpressible, and multiple causes may be observationally indistinguishable. The study will not claim general causal discovery, universal safety, or deployment-scale assurance.

Selected references

[1] W. Eiers, G. Sankaran, and T. Bultan. Quantitative Policy Repair for Access Control on the Cloud. ISSTA, 2023. doi:10.1145/3597926.3598078.

[2] K. L. Wu, C. Jenkins, S. D. Stoller, and O. Chowdhury. Automatically Tightening Access Control Policies with Restricter. TACAS, 2026. doi:10.1007/978-3-032-22752-2_6.

[3] S. Mechtaev, J. Yi, and A. Roychoudhury. DirectFix: Looking for Simple Program Repairs. ICSE, 2015. doi:10.1109/ICSE.2015.63.

[4] H. Wang, C. M. Poskitt, and J. Sun. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents. ICSE, 2026. arXiv:2503.18666.

[5] W. Eiers et al. Quantifying Permissiveness of Access Control Policies. ICSE, 2022. doi:10.1145/3510003.3510233.